

FACG — 基于 CPU 和 GPU 混合计算的频率分析

一款用于天文时间序列数据的 GPU 加速迭代预白化频率分析工具，受 [SigSpec](#) (Reegen 2007, A&A 467, 1353) 启发。

FACG 实现了完整的 SigSpec 流程——谱显著性估计、迭代预白化和全局多正弦优化——同时利用 CUDA GPU 处理计算最密集的步骤。当没有可用的 GPU 时，它会透明地回退到 CPU (NumPy)。

特性

- 基于 **CuPy** 的 **GPU 加速** —— 在 CPU 上透明回退到 NumPy
 - **NVIDIA 显卡**: 通过 CuPy/CUDA 加速
 - **Apple Silicon (M1/M2/M3...)**: 通过 PyTorch/Metal (MPS) 加速
 - 在无 GPU 或未安装相应库时，透明地回退到 CPU (NumPy)
- 兼容 **SigSpec** 的显著性公式（解析误报概率）
- 带有频率精化的迭代预白化（二分查找）
- 通过 **Levenberg-Marquardt** 算法和解析雅可比矩阵进行全局多正弦优化
- 灵活的文件命名 —— 接受任何合法的文件名（没有严格的命名规则）
- 包含 **CLI (faccg)** 和 **Python API** 的可安装 **Python** 软件包
- 处理时间报告 —— 每次迭代时间和总耗时

软件包结构

```
FACG/
├── pyproject.toml      # 软件包构建配置
├── README.md          # 英文说明文档
├── README_CN.md       # 中文说明文档
├── FACG.py            # 原始脚本（保留作参考）
├── facg/              # 可安装的 Python 软件包
│   ├── __init__.py    # 软件包入口，导出 FACGConfig 和 run_analysis
│   ├── __main__.py    # CLI 入口点: facg / python -m facg
│   ├── backend.py     # GPU/CPU 统一后端 (CuPy ↔ NumPy)
│   ├── config.py      # FACGConfig 数据类 — 所有可调参数
│   ├── spectrum.py    # 核心光谱引擎 (DFT、显著性、精化)
│   ├── optimizer.py   # 多正弦全局优化器 (GPU 雅可比矩阵 + LM)
│   ├── io.py          # 文件 I/O (读取任何文本文件，写入结果)
│   └── prewhiten.py   # 迭代预白化级联（主循环）
```

模块职责

模块	角色
backend.py	自动检测 CuPy/CUDA；提供 <code>xp</code> 、 <code>to_device()</code> 、 <code>to_host()</code> ，使得所有其他模块都与后端无关

模块	角色
spectrum.py	<code>compute_significance_spectrum()</code> — GPU 加速的批量 DFT + SigSpec 显著性公式； <code>refine_frequency()</code> — 对显著性峰值进行二分查找； <code>make_freq_grid()</code> — 频率网格构建
optimizer.py	<code>global_optimize()</code> — 使用 GPU 计算的解析雅可比矩阵同时对所有频率、振幅和相位进行 Levenberg-Marquardt 拟合
prewhiten.py	<code>run_analysis()</code> — 主级联循环：检测 → 精化 → 优化 → 减去 → 重复
config.py	包含所有用户可调参数的 <code>FACGConfig</code> 数据类，具有与 SigSpec 兼容的默认值
io.py	读取任何以空白字符分隔的文本文件；使用描述性名称写入结果 / 光谱 / 残差 / 相位图文件
__main__.py	基于 <code>argparse</code> 的 CLI，支持多个输入文件，所有参数均作为标志选项

安装

```
cd FACG
pip install -e .
```

只需一条命令即可安装全部内容。在运行时，FACG 会**自动检测**计算后端：

- 如果有支持 CUDA 的 GPU 和 CuPy 可用 → GPU 模式（并显示确认信息）。
- 如果在 Apple Silicon 芯片的 Mac 上并安装了 PyTorch → GPU 模式（通过 Metal）。
- 否则 → CPU 模式，并提示如何启用 GPU 加速：

```
△ No GPU acceleration backend found. Running on CPU (NumPy).
To enable GPU acceleration:
- For NVIDIA GPUs, install CuPy (e.g., `pip install cupy-cuda12x`).
- For Apple Silicon (macOS), install PyTorch (`pip install torch`).
```

依赖项（自动安装）：

- Python ≥ 3.9
- NumPy ≥ 1.22
- SciPy ≥ 1.8
- Pandas ≥ 1.5
- Openpyxl ≥ 3.0 (用于支持 Excel)
- Astropy ≥ 5.0 (用于支持 FITS)
- Matplotlib ≥ 3.5
- (可选，需手动安装) CuPy — 用于 GPU 加速

快速开始

命令行

```
# 分析单个文件（任何文件名都可以）
facg my_lightcurve.dat

# 在一次运行中分析多个文件
facg file1.dat file2.dat file3.dat

# 自定义参数
facg data.dat --sig-limit 6 --oversampling 40 --freq-high 50 --output-dir
./results

# 安静模式（抑制进度输出）
facg data.dat -q

# 强制使用 CPU 模式
facg data.dat --cpu

# 显示所有可用选项
facg --help
```

Python API

```
from facg import FACGConfig, run_analysis

cfg = FACGConfig(
    input_file="data.dat",
    sig_limit=5.0,
    oversampling=20.0,
    freq_high=50.0,          # 频率上限 ( $d^{-1}$ )
    output_dir="./output", # 输出目录
)
results = run_analysis(cfg)

for r in results:
    print(f"  freq = {r['freq']:12.9f}  "
          f"sig = {r['sig']:8.2f}  "
          f"amp = {r['amp']:12.9f}  "
          f"phase = {r['phase']:8.4f}  "
          f"rms = {r['rms']:12.9f}  "
          f"csig = {r['csig']:8.2f}")
```

输入格式

FACG 可以读取任何以空白字符分隔的文本文件，不需要特定的命名约定或表头。

要求	描述
分隔符	空白字符（空格或制表符）
表头	无（不期望有表头行）
列数	至少 2 列数字
第 0 列	时间戳（默认，可通过 <code>--time-col</code> 配置）
第 1 列	观测值 / 流量（默认，可通过 <code>--data-col</code> 配置）
文件名	任何有效的文件名 — 没有严格的命名规则

示例：

```
0.00000000 1.0022292321
0.02097133 1.0026197342
0.03765824 0.9967974816
0.06138463 0.9996638547
...
```

输出文件

所有输出都将写入输入文件旁边自动创建的目录 `<input_stem>_facg/` 中（如果指定了 `--output-dir`，则写入该目录）。

主结果表 — `<stem>.dat`

每个检测到的频率信号占一行。列说明：

列名	描述
<code>freq</code>	频率（每时间单位的周期数，例如 d^{-1} ）
<code>sig</code>	SigSpec 显著性（以 $\log_{10}$ 表示的误报概率）
<code>amplitude</code>	正弦分量的半振幅
<code>phase</code>	相位角（弧度）
<code>rms</code>	减去该信号及所有先前信号后的残差均方根 (RMS)
<code>csig</code>	迄今为止所有检测结果的累积显著性

中间输出

文件名	描述	控制标志
<code><stem>_spectrum_NNNN.dat</code>	第 N 次迭代的显著性 / 振幅谱	使用 <code>--no-spectrum</code> 禁用
<code><stem>_residuals_NNNN.dat</code>	第 N 次迭代的残差时间序列	使用 <code>--no-residuals</code> 禁用

文件名	描述	控制标志
<stem>_spectrum_final.dat	最终残差谱	始终写入
<stem>_residuals_final.dat	最终残差时间序列	始终写入
phase_NNNN_f*.dat	频率 f 的折叠相位图	使用 <code>--phase-diagrams</code> 启用

光谱文件包含 4 列：`freq`、`sig`、`amplitude`、`phase`。

残差文件包含 2 列：`time`、`residual`。

配置参数

所有参数都可以通过 CLI 标志或 `FACGConfig` 数据类进行设置。

频率网格

参数	CLI 标志	默认值	描述
<code>freq_low</code>	<code>--freq-low</code>	瑞利分辨率 $1/T$	频率下限
<code>freq_high</code>	<code>--freq-high</code>	奈奎斯特频率 $0.5/\Delta t$	频率上限
<code>nyquist_coeff</code>	<code>--nyquist-coeff</code>	0.5	奈奎斯特系数
<code>oversampling</code>	<code>--oversampling</code>	20.0	过采样率（与 <code>SigSpec</code> 默认值相同）

频率网格的构建方式如下：

```
freq_step = (1/T) / oversampling
grid = [freq_low, freq_low + freq_step, ..., freq_high]
```

停止条件

参数	CLI 标志	默认值	描述
<code>sig_limit</code>	<code>--sig-limit</code>	5.0	当峰值显著性低于此值时停止
<code>csig_limit</code>	<code>--csig-limit</code>	0 (禁用)	当累积显著性低于此值时停止
<code>max_iter</code>	<code>--max-iter</code>	999	最大预白化迭代次数

I/O 控制

参数	CLI 标志	默认值	描述
<code>time_col</code>	<code>--time-col</code>	0	基于 0 的时间列索引

参数	CLI 标志	默认值	描述
data_col	--data-col	1	基于 0 的数据列索引
output_dir	-o, --output-dir	<stem>_facg/	输出目录
write_spectrum	--no-spectrum	True	写入中间光谱
write_residuals	--no-residuals	True	写入中间残差
write_phase_diagram	--phase-diagrams	False	写入折叠相位图
plot	--plot	False	生成摘要图 (PNG)。默认禁用以节省时间
quiet	-q, --quiet	False	抑制进度输出

GPU 加速策略

FACG 通过 CuPy 使用 CUDA GPU 加速了两项计算密集型操作。当没有可用的 GPU 时，所有计算都会透明地回退到 CPU 上的 NumPy。

1. 显著性谱 — 批量 DFT（主要加速）

SigSpec 显著性通过直接 DFT（不是 FFT，以处理不规则的时间采样）在网格中的**每个**频率上进行评估：

$$a(f) = (1/N) \sum_i x(t_i) \cdot \cos(2\pi f t_i)$$
$$b(f) = (1/N) \sum_i x(t_i) \cdot \sin(2\pi f t_i)$$

对于 M 个频率 × N 个数据点，这被公式化为一个 (M × N) 的矩阵-向量乘积，完全在 GPU 上单次计算完成（通过分块来限制 GPU 内存使用）。

然后根据采样轮廓轴 α_0 、 β_0 和倾斜角 θ_0 (Reegen 2007) 解析计算出显著性：

$$\text{sig}(f) = N \cdot \log_{10}(e) / \sigma^2 \cdot [(a \cdot \cos\theta_0 + b \cdot \sin\theta_0)^2 / \alpha_0 + (a \cdot \sin\theta_0 - b \cdot \cos\theta_0)^2 / \beta_0]$$

典型加速比： 对于 N=5000，M=100000，比 CPU 快 10–50 倍。

2. 多正弦全局优化器 — GPU 雅可比矩阵

在每个预白化步骤之后，通过 Levenberg-Marquardt (SciPy `least_squares`) 将所有检测到的信号同时重新拟合到原始数据。模型为：

$$y(t) = \sum_k A_k \cdot \sin(2\pi f_k t + \phi_k)$$

雅可比矩阵具有解析偏导数：

$$\begin{aligned}\partial y / \partial f_k &= 2 \pi t \cdot A_k \cdot \cos (2 \pi f_k t + \varphi_k) \\ \partial y / \partial A_k &= \sin (2 \pi f_k t + \varphi_k) \\ \partial y / \partial \varphi_k &= A_k \cdot \cos (2 \pi f_k t + \varphi_k)\end{aligned}$$

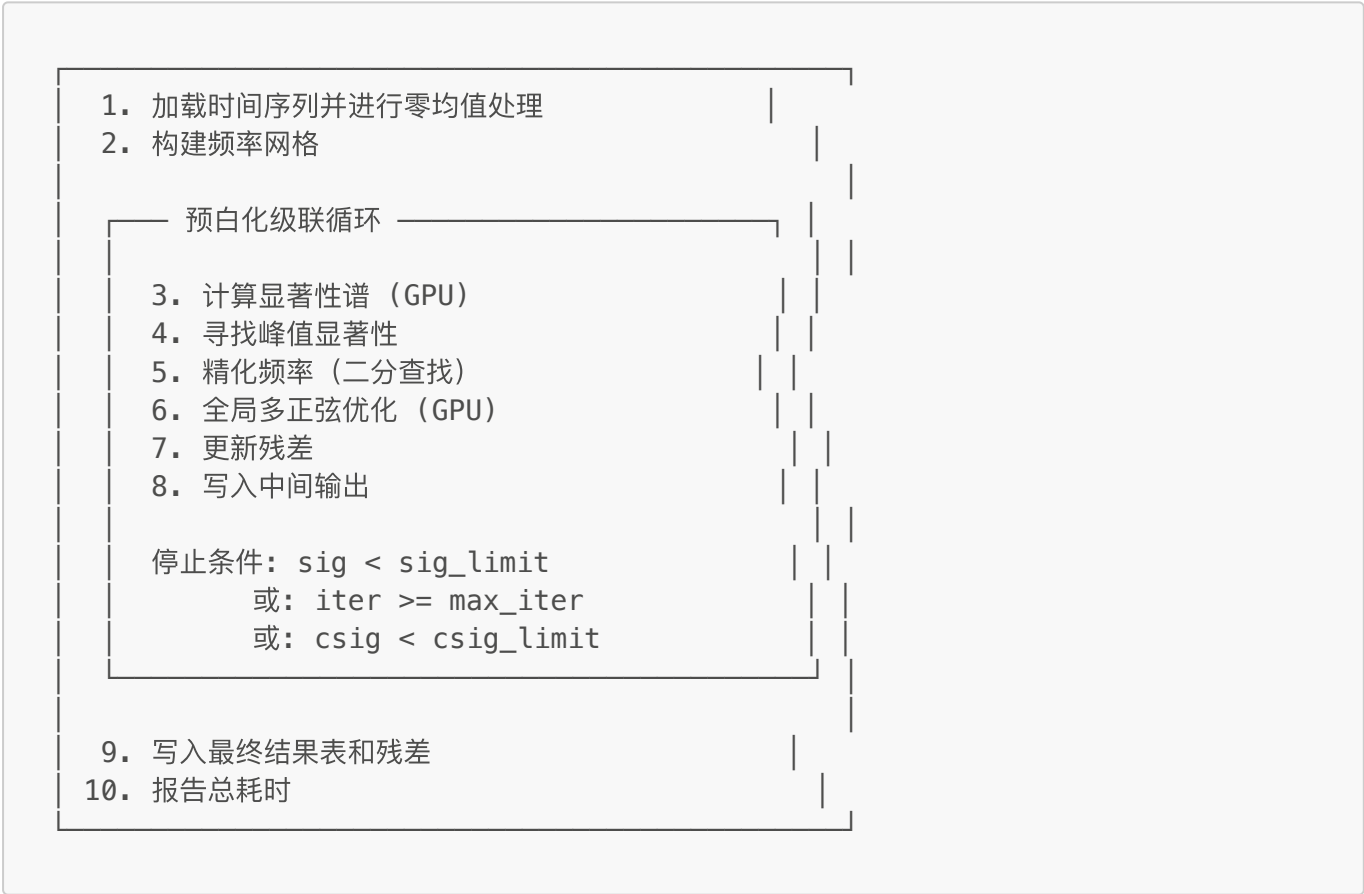
完整的 (N × 3K) 雅可比矩阵在 GPU 上一次性计算完成，然后传输到 CPU 用于 LM 求解器。这避免了 SciPy 的数值有限差分雅可比近似（每次迭代需要 6K+1 次模型评估）。

3. 频率精化 — 优化的 CPU

二分频率精化（等效于 SigSpec 的 `SigSpec_MaxSig`）在约 80 个单独的频率上评估显著性。这些规模太小，GPU 传输开销得不偿失，因此专用的 `_sig_single()` 函数使用 `np.dot()` 以获得最大 CPU 吞吐量。

算法流程

核心分析遵循与 SigSpec 相同的迭代预白化方法：



与 SigSpec 的比较

功能	SigSpec (C)	FACG (Python)
编程语言	C	Python 3
GPU 加速	否	是 (CuPy/CUDA)

功能	SigSpec (C)	FACG (Python)
显著性公式	Reegen 2007	相同
迭代预白化	是	是
全局多正弦拟合	是	是 (LM + 解析雅可比矩阵)
频率精化	二分查找 (SigSpec_MaxSig)	相同算法
累积显著性	SigSpec_CSig	相同公式
文件命名	严格 <project>/<type><iter>.dat	灵活, 任何文件名
安装	手动编译	pip install -e .
多文件支持	通过 MultiFile .ini	CLI: facg *.dat
配置	.ini 文件	CLI 标志 + Python API
时间报告	无	每次迭代 + 总计

输出示例

```
=====
FACG – Frequency Analysis of CPU and GPU mixed computing
Backend : CPU (NumPy 2.3.4)
Input   : SigSpec_Test_01.dat
Output  : SigSpec_Test_01_facg
=====

Data points   : 5000
Time base     : 99.999198
Rayleigh res  : 0.010000080
Freq step     : 0.000500004
Freq range    : [0.010000, 50.000000]
Nyquist coeff : 24.980378
Oversampling  : 20.0
# frequencies : 99981
sig threshold : 5.0
=====

iter   1: freq  34.377272957  sig  374.9393  amp  0.002558265  rms
0.002485681  [9.34s]
iter   2: freq  22.945686587  sig  450.3668  amp  0.002274437  rms
0.001901435  [9.40s]
iter   3: freq  31.915168800  sig  580.3440  amp  0.001965941  rms
0.001296536  [9.59s]
iter   4: freq  32.991913360  sig  626.2616  amp  0.001388495  rms
0.000843077  [9.51s]
iter   5: freq  10.329176329  sig  704.0327  amp  0.000962925  rms
0.000499334  [9.52s]
iter   6: max sig = 3.9187 < 5.0 – stopping.
=====

Detected frequencies : 5
```

```
Total elapsed time    : 66.422 s
=====
```

作者

Niu Hubiao

许可证

MIT